MarketPulse AI

Consumer Consumption Intelligence & Spatial Analytics Platform for Indian Regional Markets

Domain: FMCG & Retail Analytics	**Core Stack:** React 19 • TypeScript • Vite	**Design:** Neo-Brutalist System	**Audience:** Academic / Viva / Technical Review

1. Project Title

Verified System Name: MarketPulse AI (derived from package.json, design.md, index.html, and Navbar.tsx).

Recommended Academic Title: MarketPulse AI — Consumer Consumption Intelligence & Spatial Analytics Platform for Indian Regional Markets.

2. Project in One Paragraph

EXECUTIVE BRIEF FOR TEACHER / EVALUATOR

MarketPulse AI *is an interactive, web-based market intelligence and consumer analytics platform designed to analyze, visualize, and benchmark regional consumer product consumption behaviors across India. In fast-moving consumer goods (FMCG), electronics, apparel, and personal care, decision-makers often struggle to understand hyper-local market dynamics because aggregate national data masks state- and city-level differences. Built for market research analysts, brand managers, and retail strategists, this application lets users drill down geographically from an all-India view into 11 specific states and 44 major cities through interactive SVG and geospatial Leaflet maps. Users can slice and dice data across 4 core retail categories, 16 brands, and 64 product SKUs, apply multi-tier demographic filters (age groups, gender, seasonal quarters, and years), run side-by-side comparative analyses between competing regions or brands, inspect detailed SKU profiles in modals, and export audit-ready reports in CSV, Excel-compatible sheets, or print-ready PDFs. The primary purpose is to provide business analysts with sub-second, multi-dimensional market visibility through an engaging, high-contrast Neo-Brutalist dashboard.*

3. Problem Statement

Existing Problem: Consumer demand across India is extraordinarily diverse. Purchasing power, seasonal demand velocity, and demographic affinities vary sharply between tier-1 and tier-2 cities. National aggregate sales metrics hide localized demand dips and festive spikes (e.g., Diwali in Q4).

Why the Problem Matters: Brands, distributors, and supply-chain planners risk misallocating warehouse inventories, overspending on ineffective regional marketing, and losing shelf market-share when lacking localized intelligence.

Limitations of Traditional Approaches: Static PDF decks, disconnected spreadsheets, and legacy BI suites are sluggish, lack intuitive spatial context, and require laborious pivot table re-queries to extract multi-dimensional demographic answers.

How MarketPulse AI Solves This: Combines high-speed client-side aggregations with a tactile Neo-Brutalist visual interface, multi-tier cascading dropdown filters, an interactive India SVG map, geospatial Leaflet markers, and side-by-side benchmarking to deliver sub-second market visibility.

4. Objectives

1. **To deliver interactive geographic drilldown:** Enable seamless spatial exploration from an all-India view into 11 states and 44 metropolitan cities.
2. **To compute executive business KPIs:** Provide instant calculations for Total Revenue (Cr/Lakhs), Unit Sales, Mean Market Share %, Consumer Footfall, and YoY Growth Rate.
3. **To enable multi-dimensional filtering:** Allow simultaneous segmentation across State, City, Category, Brand, SKU, Year (2024–2026), Quarter, Month, Age Cohort, and Gender.
4. **To facilitate head-to-head competitive benchmarking:** Deliver side-by-side comparisons between states, cities, categories, brands, or products with automated margin delta indices.

5. **To provide SKU-level product catalog intelligence:** Surface detailed product cards with drilldown modals highlighting top-5 contributing states and cities.

6. **To support audit-ready multi-format data export:** Equip enterprise analysts with client-side CSV downloads, Excel-compatible (.xls) files, and print-ready styled PDFs.

7. **To implement a bespoke Neo-Brutalist design language:** Combine high-contrast solid borders, flat offset box shadows, Google typography, and Framer Motion micro-interactions.

5. What Has Actually Been Built (Implementation Audit)

A rigorous inspection of the physical repository reveals the following concrete implementation status across all technical layers:

Component / Layer	Status	Implementation Details in Codebase
Landing Page	**Implemented**	Hero section, animated SVG connection paths, feature showcases, preview panels, and responsive navigation in LandingPage.tsx.
Layout & Navigation	**Implemented**	Sticky top navbar, mobile drawer, desktop sidebar menu, and active link routing in MainLayout.tsx, Navbar.tsx, Sidebar.tsx.
Geographic Drilldown	**Implemented**	Interactive SVG vector map of India with custom state hitboxes, hover cards, and 4-step wizard navigation in HomeMapPage.tsx & IndiaSvgMap.tsx.
Geospatial Leaflet Map	**Implemented**	Leaflet + React-Leaflet integration with custom DivIcon circle markers for 44 city coordinates, revenue popups, and zoom levels in LeafletMap.tsx.
Analytics Dashboard	**Implemented**	Real-time KPI summaries, 10-parameter filter matrix, Recharts area/bar/pie charts, SKU revenue rankings, and sub-second simulated query lag in DashboardPage.tsx.
Comparison Engine	**Implemented**	Multi-type comparison selector, automated delta margin calculation, dual-line historical revenue charts, and side-by-side SKU rankings in ComparePage.tsx.
Products Catalog	**Implemented**	Searchable product cards with category-themed badges, YoY growth indicators, and drilldown detail modal dialogs in ProductsPage.tsx.
Reports & Exporters	**Implemented**	Paginated table with multi-column sorting, search filters, client-side CSV generator, Excel .xls MIME-type export, and print media CSS in ReportsPage.tsx & Table.tsx.
Deterministic Data Engine	**Implemented**	Seeded pseudo-random generator producing ~2,500+ structured records factoring in seasonality, state weights, and category multipliers in mockDataGenerator.ts.
Analytical Query Service	**Implemented**	In-memory filtering, KPI mathematical reductions, trend grouping, ranking, and distribution logic in dataService.ts.
Neo-Brutalist System	**Implemented**	Atomic components (Button, Card, Badge, Input, Dropdown, Modal, EmptyState, LoadingSkeleton) with @theme tokens and offset drop-shadow utilities in index.css.
User Profile Settings	**Partial**	Editable profile form in SettingsPage.tsx saves to component state with simulated success alert; API endpoint dropdown is locked to 'mock'.
Authentication / RBAC	**Not Implemented**	No user login, registration, JWT handling, session management, or route guards exist. All pages are publicly accessible.
Backend Web Server	**Not Implemented**	No live backend server exists in code. SettingsPage.tsx explicitly notes: 'Switch Environment to dev once FastAPI backend container is operational.'
Physical Database	**Not Implemented**	No relational (PostgreSQL/MySQL) or NoSQL (MongoDB) database is connected. All records exist in client-side memory cache.
AI / ML Model Service	**Not Implemented**	No machine learning pipelines or LLM inference scripts exist. 'AI' is branding; metrics use deterministic mathematical formulas.

6. Key Features Implemented

1. Interactive Geographic Drilldown Portal

• **What it does:** Provides a visual vector map of India with 11 commercial states. Hovering displays quick KPI previews; clicking selects the state and loads available cities for focused drilldown.

• **How it works:** Implemented in IndiaSvgMap.tsx using interactive SVG coordinate hitboxes, Framer Motion animations, and real-time state KPI reductions.

• **User benefit:** Gives non-technical users an engaging spatial entry point rather than tedious text dropdowns.

2. Geospatial City Hotspots via Leaflet

• **What it does:** Visualizes city-level consumption hotspots across real geographical latitude/longitude coordinates with interactive pan/zoom controls.

• **How it works:** Implemented in LeafletMap.tsx with react-leaflet, OpenStreetMap raster tiles, custom L.DivIcon HTML markers, and revenue popups for 44 cities.

• **User benefit:** Enables regional distribution managers to immediately spot spatial demand clusters and supply gaps.

3. Multi-Tier Enterprise Filter Matrix

• **What it does:** Allows concurrent filtering across 10 dimensions: State, City, Category, Brand, SKU, Year (2024–2026), Quarter, Month, Age Group, and Gender.

• **How it works:** Implemented in DashboardPage.tsx and dataService.ts with cascading resets (e.g. changing category resets brand and SKU; changing state resets city).

• **User benefit:** Allows analysts to drill into granular target segments (e.g. 'Apple iPhone in Mumbai during Q4 2025 among 25–34 year olds').

4. Real-Time Executive KPI Metric Cards

• **What it does:** Computes and displays 5 top-line metrics: Total Revenue (Cr/Lakhs), Total Units Sold, Mean Market Share %, Consumer Footfall Count, and YoY Growth Rate.

• **How it works:** Calculated via single-pass mathematical reduction in calculateKPIs() inside dataService.ts.

• **User benefit:** Delivers an immediate executive health check under any active filter combination.

5. Neo-Brutalist Visual Analytics Suite

• **What it does:** Displays 4 responsive charts: Monthly Revenue Area Chart, Category Sector Split Donut, Top-10 Brands Bar Chart, and Demographic Age/Gender Splits.

• **How it works:** Custom-styled wrappers around recharts primitives with thick borders (#111111), Space Grotesk fonts, and custom tooltip components.

• **User benefit:** Translates multi-thousand record tables into high-impact visual trends.

6. Head-to-Head Comparison Intelligence Engine

• **What it does:** Enables direct side-by-side benchmarking across 5 entity types: State vs. State, City vs. City, Brand vs. Brand, Category vs. Category, or Product vs. Product.

• **How it works:** Implemented in ComparePage.tsx and getComparisonData(). Calculates percentage delta margins and merges chronological series for dual-line graphs.

• **User benefit:** Eliminates manual spreadsheet cross-referencing when comparing rival brands or geographic hubs.

7. Product Catalog with SKU Drilldown Modal

• **What it does:** Lists all products in the database with search and category filters. Clicking 'View Drilldown' opens an animated modal showing volume, market share, and top-5 contributing states and cities.

• **How it works:** Implemented in ProductsPage.tsx using Framer Motion modal dialogs and regional revenue group-by algorithms.

• **User benefit:** Allows SKU product managers to identify regional strongholds for individual products.

8. Enterprise Reports Ledger & Multi-Format Exporters

• **What it does:** Searchable, sortable table of consumption records with one-click export to CSV, simulated tab-delimited Excel (.xls), or print-styled PDF.

• **How it works:** Implemented in ReportsPage.tsx and Table.tsx using client-side data URIs and print media CSS (@media print).

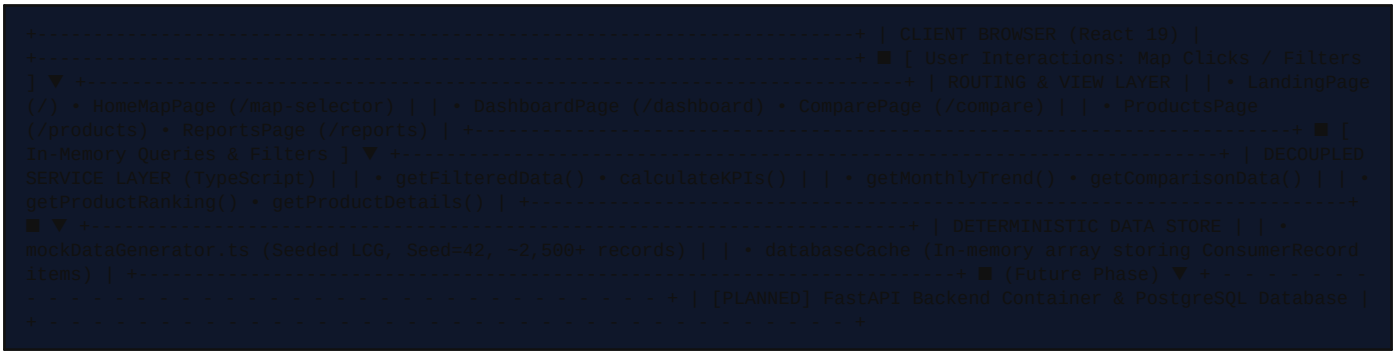
• **User benefit:** Enables analysts to extract clean data for stakeholder presentations or offline auditing.

7. Technology Stack

Category	Technology	Purpose in Project
Frontend Framework	React 19.2.8	Component-based reactive user interface architecture
Programming Language	TypeScript 6.0.2	Compile-time type safety across models, props, and analytical services
Build Tool / Bundler	Vite 8.2.0	Fast ESM development server and production bundler
CSS Framework	Tailwind CSS 4.3.3	Utility styling, design system tokens via @theme
Design Philosophy	Neo-Brutalism	High-contrast #111111 borders, flat offset shadows, bright card fills
Typography	Space Grotesk & Poppins	Display heading and UI body fonts loaded via Google Fonts CDN
Routing	React Router DOM 7.18.2	Client-side SPA route navigation and search param sync
Charting Engine	Recharts 3.10.1	Area, Bar, Pie, and Line charts wrapped in Neo-Brutalist styles
Geospatial Maps	Leaflet 1.9.4 / React-Leaflet 5.0.0	Interactive tile map plotting 44 Indian city coordinates
Vector Map	SVG + React DOM	Interactive stylized vector map of India with state boundaries
Animations	Framer Motion 12.43.0	Staggered entrances, floating hero graphics, animated SVG paths, modals
Icon Library	Lucide React 1.28.0	Vector iconography styled at stroke-[2.5] and stroke-[3]
Linter	Oxlint 1.75.0	High-performance Rust-based linter enforcing clean React code
Data Storage	Client In-Memory Cache	Deterministic generator producing ~2,500+ structured consumer records
Backend / API	Decoupled Mock Service	Architecture prepared for future FastAPI container integration

8. System Architecture

MarketPulse AI follows a **Client-Side Single Page Application (SPA)** architecture using a **Decoupled Service Pattern**. All business, filtering, and mathematical logic is isolated inside dataService.ts, completely separating presentation components from the underlying data layer.



9. Complete Project Workflow

1. **User opens application:** User lands on the public Landing Page (/), reviewing value propositions, live previews, and feature highlights.
2. **User initiates geographic discovery:** Clicks 'Explore India Map', navigating to the 4-step Map Portal (/map-selector).
3. **User selects state on SVG map:** Hovering previews state metrics; clicking selects an active state (e.g. *Maharashtra*).
4. **User selects city:** The portal reveals available metropolitan hubs (e.g. *Mumbai*). User locks selection and clicks 'Launch Intelligence Dashboard'.
5. **System synchronizes parameters:** App redirects to /dashboard?state=Maharashtra&city=Mumbai. The dashboard reads URL search parameters to initialize the filter state.
6. **Dashboard calculates metrics:** The decoupled service filters records in memory, rendering 5 top KPI cards, monthly revenue area charts, sector pie charts, brand rankings, and Leaflet city pins.
7. **User applies multi-tier filters:** User changes category, brand, year, or demographic cohort. A simulated 300ms query lag renders skeleton cards, followed by instant recalculation.
8. **User runs side-by-side comparison:** User navigates to /compare, selects 'City vs City' (e.g. *Mumbai* vs. *Bangalore*), and reviews margin differentials and dual timeline curves.
9. **User inspects product catalog:** User navigates to /products, searches for an SKU (e.g. *'iPhone 15'*), and clicks 'View Drilldown' to open a modal with top-5 regional breakdowns.

10. **User exports reports:** User navigates to /reports, filters raw records, and clicks 'Excel' to download a spreadsheet or 'Print/PDF' to trigger a formatted PDF document.

10. Database Analysis

Database Technology: In-Memory Client-Side Simulated Database. No physical SQL or NoSQL database engine exists in the current build.

Primary Data Schema: ConsumerRecord interface defined in src/types/index.ts.

Field	Type	Description	Example Value
id	string	Unique record identifier	'MP-1042'
state / city	string	Geographic hierarchy	'Maharashtra' / 'Mumbai'
brand / category	string	Commercial hierarchy	'Apple' / 'Electronics'
product	string	SKU name	'iPhone 15'
revenue / sales	number	Total revenue (INR) and units sold	1,250,000 / 28
consumerCount	number	Count of purchasing consumers	25
marketShare / growth	number	Category share % and YoY growth %	18.4 / 14.2
date / month / year	string / number	Calendar temporal attributes	'2025-10-15' / 'October' / 2025
quarter	string	Financial quarter	'Q4'
ageGroup / gender	enum	Target shopper demographics	'25-34' / 'Female'

11. API Analysis

Backend REST APIs: Currently unattached. SettingsPage.tsx contains a simulated connection panel pointing to http://localhost:8000/api/v1 with environment locked to 'mock'.

Planned Endpoints: GET /api/v1/records (filtering), GET /api/v1/kpis (KPI reductions), GET /api/v1/compare (benchmarking), and GET /api/v1/products/:name (SKU drilldowns).

External Web Services Used:

- **OpenStreetMap Tile Server:** https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png provides geographic map tiles to Leaflet without authentication.
- **Google Fonts CDN:** Serves the Space Grotesk and Poppins fonts declared in index.html.

12. AI / ML Analysis

Implementation Status: Not implemented in code. The term 'AI' in 'MarketPulse AI' is platform branding.

Honest Technical Details: The repository does not contain machine learning models, training scripts, or LLM API calls. Growth metrics and market share predictions are calculated deterministically using mathematical weights (e.g. Diwali festive multiplier of 1.45x in Oct/Nov; metro city multipliers of 1.4x) inside mockDataGenerator.ts. A production machine learning forecasting pipeline (such as ARIMA or Prophet) is planned for future backend development.

13. Folder Structure

The project follows a clean, modular React + TypeScript architecture:

- src/charts/: Recharts wrapper components styled with Neo-Brutalist design tokens (NeoAreaChart, NeoBarChart, NeoPieChart, NeoLineChart, CustomTooltip).
- src/components/: Reusable UI library (Button, Card, Badge, Input, Dropdown, Modal, EmptyState, LoadingSkeleton, Table, Navbar, Sidebar).
- src/constants/: Domain constants (STATES_AND_CITIES, CATEGORIES_BRANDS_PRODUCTS, YEARS, QUARTERS, MONTHS).
- src/layouts/: MainLayout shell coordinating top navbar, sidebar drawer, and dynamic outlet content.
- src/maps/: Geographic components (IndiaSvgMap for vector drilldowns; LeafletMap for geospatial city coordinates).
- src/pages/: High-level page views (LandingPage, HomeMapPage, DashboardPage, ComparePage, ProductsPage, ReportsPage, AboutPage, SettingsPage).

- src/routes/: React Router configuration routing URLs to page components.
- src/services/: Isolated business logic layer (dataService.ts) containing filtering and mathematical aggregation algorithms.
- src/types/: Central TypeScript contracts and data models.
- src/utils/: Synthetic seeded data generation engine (mockDataGenerator.ts).

14. Important Code Modules

- **dataService.ts:** Analytical brain of the system. Contains all filtering algorithms, KPI mathematical reductions, and comparison group-by processors. Completely decouples logic from UI.
- **mockDataGenerator.ts:** Deterministic data generation engine. Uses a seeded LCG (seed=42) and economic/seasonal multipliers to generate ~2,500+ realistic transaction records.
- **DashboardPage.tsx:** Primary operational view. Coordinates 10-parameter filter drawer, 5 KPI cards, 4 Recharts visual components, and the Leaflet spatial map.
- **ComparePage.tsx:** Side-by-side benchmarking engine. Computes percentage delta differentials between competing entities and merges timeline series onto a shared multi-line chart.
- **IndiaSvgMap.tsx:** Custom interactive SVG map of India. Features interactive state hitboxes, real-time KPI card hover previews, and Framer Motion micro-interactions.
- **LeafletMap.tsx:** Geospatial mapping component. Maps 44 Indian cities to geographical coordinates using OpenStreetMap tiles and custom HTML circle markers.
- **Table.tsx:** Enterprise generic table component (Table). Implements multi-key searching, column sorting, pagination, and client-side CSV downloads.

15. Security Assessment

- **Authentication & Authorization:** None implemented. All routes are public.
- **Password & Token Handling:** None required as there are no user accounts or JWT sessions.
- **Secrets & Credentials:** Clean. No private API keys or tokens are stored in source code.
- **Identified Security Weaknesses for Production:** Client-side data exposure (all ~2,500 records reside in browser RAM), lack of route guards, and lack of backend rate limiting. Full enterprise security (JWT, RBAC) must be added when binding to the FastAPI backend.

16. Error Handling & Validation

- **Empty State Handling:** Implemented via EmptyState.tsx. Displays an alert card with 'Reset All Filters' if zero records match queries.
- **Loading Skeletons:** Implemented via LoadingSkeleton.tsx. Displays animated pulse cards during simulated 300ms query transitions.
- **Cascading Filter Resets:** Selecting a new Category resets Brand and SKU; changing State resets City to prevent invalid queries.
- **Type Safety:** Strict TypeScript types and optional chaining prevent runtime null pointer exceptions.

17. Testing

- **Automated Tests:** No automated test files (Jest/Vitest/Playwright) are currently present.
- **Static Quality:** Oxlint (.oxlintrc.json) and TypeScript strict type checking (tsc -b) enforce clean syntax and type adherence.
- **Manual Testing:** Verified functional behavior across all 8 routes, filter combinations, map markers, modals, and file exports.

18. Current Project Status: Functional Application / High-Fidelity MVP

The project is a fully functional, interactive frontend Single Page Application. It is not merely a static mockup; all navigation, filters, maps, charts, mathematical aggregations, and file exports are operating smoothly with in-memory data. It is ready for backend integration.

19. Technical Strengths

- **Striking Neo-Brutalist UI:** Distinctive design with solid 3px borders, flat offset shadows, and Framer Motion micro-animations.
- **Decoupled Architecture:** Clean separation of business logic in dataService.ts allows rapid backend binding.

- **Dual-Mapping Strategy:** Combines an artistic SVG India map with a precision Leaflet geospatial map.
- **Sub-Second Query Velocity:** Fast in-memory array filtering and memoization eliminate UI latency.
- **Zero-Dependency Multi-Format Exporters:** Native CSV downloads, Excel .xls sheets, and print-ready PDF styling.

20. Realistic Limitations

- **Client-Side RAM Bounds:** In-memory storage cannot scale to millions of transaction rows without server-side pagination.
- **Synthetic Data:** Lacks live integrations with real retail POS or e-commerce webhooks.
- **No Machine Learning:** Growth metrics are generated mathematically rather than via trained predictive ML models.
- **Lack of Session Persistence:** Settings changes reset on page reload.

21. Future Scope

- **Short-Term:** Add localStorage persistence for user preferences, add Vitest unit test suites for dataService.ts, and expand dataset to all 28 Indian states.
- **Advanced:** Build the planned FastAPI Python backend with PostgreSQL database and integrate JWT authentication with role-based permissions.
- **Production:** Connect true ML forecasting models (ARIMA / Prophet) in Python to predict future seasonal consumption and containerize with Docker.

22. Teacher Explanation (1–2 Minute Spoken Script)

DIRECT VIVA SCRIPT (SPEAK TO TEACHER)

*"Good morning/afternoon, Sir/Ma'am. Today I am presenting **MarketPulse AI**, a consumer market intelligence and spatial analytics platform focused on regional Indian markets.*

***The Problem:** In India's retail and FMCG sectors, consumer behavior varies drastically between states and cities. National sales averages often hide important local trends—such as festive Diwali sales peaks in northern states or distinct category preferences in southern metros. Traditional reports are static and slow to filter.*

***What I Built:** I built a fully interactive analytics platform using React 19, TypeScript, Vite, and Tailwind CSS, styled with a modern Neo-Brutalist design language. The application features a 4-step geographic drilldown portal using an interactive SVG map of India and an interactive Leaflet geospatial map plotting 44 major cities.*

***How It Works:** In the dashboard, users can filter data across 10 dimensions—including states, cities, categories, brands, SKUs, years, quarters, age groups, and genders. My decoupled service layer processes these queries in memory to instantly calculate top-level KPIs, monthly revenue trends, category splits, and brand rankings using Recharts.*

***Key Features:** Beyond the dashboard, I built a side-by-side comparison engine that calculates performance margin deltas between competing cities or brands, a product catalog with SKU detail modals, and an export center that generates CSV files, Excel spreadsheets, and print-ready PDF reports.*

***Architecture:** The frontend is completely decoupled from the data layer. While the current version runs on a deterministic in-memory data engine with over 2,500 records, the service architecture is specifically designed to connect directly to a FastAPI backend in future phases. Thank you!"*

23. Technical Viva Questions & Answers

Q1: What is the primary purpose of MarketPulse AI?

Answer: It is an interactive market intelligence dashboard designed to analyze and visualize regional consumer product consumption across 11 Indian states and 44 cities, breaking down trends by demographic cohorts, product SKUs, and seasonal quarters.

Q2: What frontend technology stack did you use, and why?

Answer: I used React 19 with TypeScript and Vite. React provides a reactive component model; TypeScript guarantees compile-time type safety; and Vite offers ultra-fast build times and Hot Module Replacement (HMR).

Q3: What is Neo-Brutalism in the context of your UI?

Answer: Neo-Brutalism is a modern design aesthetic characterized by high contrast, thick solid #111111 borders, flat offset drop shadows (without blur), vibrant accent colors, bold uppercase typography (Space Grotesk), and tactile button-press micro-animations.

Q4: Does this project have a live backend server?

Answer: Not in the current version. The application runs as a client-side SPA with a decoupled service layer (dataService.ts) querying an in-memory cache. It is architected to easily connect to a FastAPI REST backend in the future.

Q5: Does the project connect to a physical database?

Answer: No. Data is generated dynamically using a seeded pseudo-random generator (mockDataGenerator.ts) and cached in memory.

Q6: How does the deterministic data generator work?

Answer: It uses a Linear Congruential Generator with a fixed seed of 42. This ensures that every time the app runs, it produces the exact same realistic dataset (~2,500+ records) with consistent seasonal spikes (e.g., Diwali in Q4) and state economic weights.

Q7: What libraries did you use for charting and data visualization?

Answer: I used Recharts to build Area, Bar, Pie, and Line charts, customized with Neo-Brutalist borders, custom tooltips, and Rupee currency formatters.

Q8: How do the two mapping components differ?

Answer: IndiaSvgMap is a custom, lightweight SVG-based vector map used for rapid state-level selection. LeafletMap is a real geospatial mapping component using OpenStreetMap tiles and geographical latitude/longitude coordinates to plot 44 specific city markers.

Q9: How are dependent dropdown filters handled?

Answer: In DashboardPage.tsx, state changes trigger cascading updates. If the user changes State, City is reset. If Category changes, Brand and Product are reset to ensure queries never reference invalid combinations.

Q10: How do you achieve sub-second filtering performance without a database index?

Answer: The dataset of ~2,500 records is stored in browser RAM. Using React's useMemo hook, single-pass array filter and reduce operations execute in just a few milliseconds.

Q11: How does the Side-by-Side Comparison feature calculate deltas?

Answer: It queries Entity A and Entity B separately using shared filters, computes the percentage difference using ((valB - valA) / valA) * 100, and merges their timeline series chronologically onto a shared multi-line Recharts graph.

Q12: How is the Excel export implemented without a backend?

Answer: In ReportsPage.tsx, columns and rows are converted into tab-delimited text and formatted as a data:application/vnd.ms-excel data URI, which the browser downloads natively as an .xls file.

Q13: How does the print-ready PDF export work?

Answer: It triggers window.print() and uses Tailwind's print: media queries (e.g., print:hidden on navbar, sidebar, and filter inputs) so the browser prints only the official, timestamped report ledger.

Q14: Is there any real Artificial Intelligence or Machine Learning in the project?

Answer: No. The term 'AI' is part of the platform's branding. Growth percentages and seasonal demand shifts are computed using mathematical weighting algorithms rather than machine learning models.

Q15: Why is Axios present in package.json if it isn't used?

Answer: Axios was installed during initial scaffolding to prepare for connecting to the planned FastAPI backend. In the current build, all data is served internally via dataService.ts.

Q16: How is routing handled?

Answer: Routing is managed by react-router-dom (version 7) using a root BrowserRouter. Page views are organized under a shared MainLayout, with wildcards redirecting back to /.

Q17: What security mechanisms are currently implemented?

Answer: The current client-side build does not implement authentication, password hashing, or route guards. No private API keys are exposed. Full enterprise security (JWT, RBAC) is planned for the backend integration phase.

Q18: What happens if a user applies filters that match zero records?

Answer: The UI renders an EmptyState component displaying a warning card with an alert icon and a 'Reset All Filters' button.

Q19: What is Oxlint?

Answer: Oxlint is an ultra-fast, Rust-based JavaScript/TypeScript linter configured in .oxlintrc.json to enforce clean code and catch syntax errors.

Q20: What is the primary limitation of the current architecture?

Answer: Scalability. Because all records are held in browser memory, the application cannot handle millions of rows without a dedicated backend server and database.

Q21: What would be your immediate next step to take this project to production?

Answer: Develop the FastAPI backend with a PostgreSQL database, replace dataService.ts calls with real Axios API requests, and add JWT-based user authentication.

24. Quick Project Cheat Sheet

Parameter	Summary Details
Project Name	MarketPulse AI
Problem Solved	Fragmented Indian regional consumer demand; national aggregates mask localized trends.
Solution	Interactive spatial and demographic analytics dashboard with multi-tier drilldowns and benchmarking.
Frontend Stack	React 19, TypeScript, Vite, Tailwind CSS v4, Framer Motion, Lucide React.
Backend Stack	Decoupled client architecture ready for FastAPI integration (none in active code).
Database	In-memory cached array of ~2,500+ records generated deterministically (seed=42).
AI / ML Status	Branding only; metrics calculated via mathematical seasonality and state economic multipliers.
APIs Used	OpenStreetMap tile server (Leaflet) and Google Fonts CDN. No live backend REST endpoints.
Authentication	None (publicly accessible frontend MVP).
Main Features	Dual maps (SVG + Leaflet), 10-tier filter matrix, real-time KPIs, Recharts analytics, side-by-side compare, SKU modals, CSV/Excel/PDF export.
Architecture	Client-Side SPA with an isolated Decoupled Service Layer pattern.
Current Status	Functional Application / High-Fidelity MVP.
Key Technical Highlight	Sub-second multi-dimensional analytical queries on ~2,500+ records with zero-dependency multi-format exports in Neo-Brutalism styling.